

# Documentación completa — Proyecto NetworkMonitor y Análisis con Data Science

---

Esta documentación explica **de principio a fin** todo lo implementado en el proyecto: conceptos, técnicas, archivos y directorios, para que puedas entender qué hace cada parte y por qué.

---

## Índice

1. [Objetivo del proyecto](#)
  2. [Estructura de directorios y archivos](#)
  3. [Parte 1: Captura de tráfico en Java \(NetworkMonitor\)](#)
  4. [Parte 2: Análisis en Python](#)
  5. [Técnicas de Data Science utilizadas](#)
  6. [Gráficas generadas](#)
  7. [Flujo completo de datos](#)
  8. [Cómo ejecutar todo](#)
  9. [Glosario de términos](#)
- 

## 1. Objetivo del proyecto

El proyecto cumple una **práctica de monitoreo y análisis de tráfico de red** con dos grandes bloques:

- **Captura:** Un programa en **Java** (Apache NetBeans + Swing) que captura paquetes de red en tiempo real en una interfaz Wi-Fi, los muestra en una tabla tipo Wireshark y los exporta a archivos **CSV**.
- **Análisis:** Scripts en **Python** que cargan esos CSV, limpian los datos, aplican **técnicas de Data Science** (clasificación KNN, clustering K-Means, análisis predictivo) y generan **gráficas** y un **reporte de conclusiones**.

Los requisitos vienen descritos en [readme.md](#), [readme1.md](#) y [readme2.md](#) (en la raíz del proyecto).

---

## 2. Estructura de directorios y archivos

```
NetworkMonitor/
├─ pom.xml           # Configuración Maven del proyecto Java
├─ readme.md         # Enunciado de la práctica
├─ readme1.md        # Resumen de análisis y visualización
├─ readme2.md        # Requisitos adicionales (MAC, filtros,
evidencia)
├─ DOCUMENTACION.md # Este archivo – documentación completa
├─ EVIDENCIA.md      # Evidencia de entrega (flujo, ejemplo CSV,
gráficas)
├─ captura_red.csv   # CSV de ejemplo (o captura_red_YYYY-MM-
DD.csv)
└─
```

```

├─ src/main/java/com/mycompany/networkmonitor/
│   └─ NetworkMonitor.java          # Programa principal Java: captura, tabla,
filtros, exportación CSV
├─ analisis_python/                # Proyecto de análisis en Python
│   ├── requirements.txt            # Dependencias: pandas, numpy, scikit-learn,
matplotlib, seaborn, jupyter
│   ├── limpieza_datos.py          # Carga CSV, limpieza, codificación, columnas
derivadas
│   ├── modelos.py                 # KNN, K-Means, regresión logística
│   ├── visualizacion.py           # Generación de todas las gráficas PNG
│   ├── main.py                    # Script principal: ejecuta todo sin Jupyter
│   └─ analisis_trafico.ipynb      # Notebook Jupyter (opcional) con el mismo
análisis por secciones
│   └─ CONCLUSIONES.md             # Reporte final con resultados e
interpretación
│   └─ resultados/                 # Carpeta donde se guardan las gráficas PNG
(01 a 09)
│   ├── 01_tcp_vs_udp.png
│   ├── 02_dispersion_tamano_hora.png
│   ├── 03_puertos_frecuentes.png
│   ├── 04_clusters_equipos.png
│   ├── 05_trafico_por_hora.png
│   ├── 06_confusion_knn.png
│   ├── 07_prediccion_protocolo_hora.png
│   ├── 08_servicios_frecuentes.png
│   └─ 09_apps_inferidas.png

```

### 3. Parte 1: Captura de tráfico en Java (NetworkMonitor)

#### 3.1 Conceptos necesarios

- **Paquete de red:** Unidad de datos que viaja por la red. Cada fila en la tabla (y en el CSV) corresponde a un paquete capturado.
- **IP (Internet Protocol):** Dirección lógica de un equipo en la red (ej. **10.20.53.78**). Se distingue **IP origen** (quien envía) e **IP destino** (quien recibe).
- **MAC (Media Access Control):** Dirección física de la tarjeta de red, única por dispositivo (formato **aa:bb:cc:dd:ee:ff**). En Wi-Fi promiscuo a veces se ve la MAC del punto de acceso.
- **Puerto:** Número (0–65535) que identifica un servicio o aplicación en un equipo. Por ejemplo: 443 = HTTPS, 53 = DNS.
- **TCP y UDP:** Protocolos de transporte. TCP es orientado a conexión (web, correo); UDP es sin conexión (DNS, streaming en tiempo real).
- **Pcap4J:** Librería Java que permite capturar paquetes en vivo usando los controladores del sistema (Npcap en Windows, libpcap en Linux/Mac). Sin ella, Java no puede “escuchar” el tráfico de red directamente.
- **Swing:** Conjunto de componentes gráficos de Java (ventanas, tablas, botones) con los que se construye la interfaz del monitor.

#### 3.2 Archivo: `src/main/java/com/mycompany/networkmonitor/NetworkMonitor.java`

Es la **única clase Java** del proyecto. Hace lo siguiente:

| Sección                | Qué hace                                                                                                                                                                                                                                                                                                                                                                            |
|------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Constructor</b>     | Crea la ventana (JFrame), la tabla con columnas (IP Origen, IP Destino, MAC Origen, MAC Destino, Puerto Ori, Puerto Des, Protocolo, Longitud, Hora Captura, Equipo Local, SSID), los campos de texto (Equipo local, SSID Wi-Fi), el panel de <b>filtros</b> (Protocolo, IP, Puerto destino) y los botones Iniciar/Detener/Exportar.                                                 |
| <b>Filtros</b>         | Usa <code>TableRowSorter</code> sobre el modelo de la tabla. "Aplicar filtros" restringe las filas visibles según protocolo (TCP/UDP), texto en IP o puerto destino. "Limpiar filtros" quita el filtro. La exportación CSV <b>siempre</b> exporta <b>todas</b> las filas del modelo (no solo las filtradas).                                                                        |
| <b>startCapture()</b>  | Muestra un diálogo para elegir la interfaz de red ( <code>Pcaps.findAllDevs()</code> ), abre el adaptador en modo promiscuo y lanza un hilo que en bucle llama a <code>handle.getNextPacket()</code> y a <code>processPacket()</code> .                                                                                                                                             |
| <b>processPacket()</b> | Por cada paquete: si es IPv4 y TCP o UDP, extrae IP origen/destino, puertos, protocolo, longitud; si existe cabecera Ethernet, extrae MAC origen y destino; obtiene hora actual (HH:mm:ss), nombre de equipo y SSID de los campos de la interfaz; añade una fila al <code>DefaultTableModel</code> . Los paquetes que no son IPv4+TCP/UDP se descartan (no se añaden filas vacías). |
| <b>exportCSV()</b>     | Genera un archivo con nombre <code>captura_red_YYYY-MM-DD.csv</code> (fecha del día actual). Escribe la primera línea con los nombres de las columnas del modelo y luego todas las filas. Así se puede exportar un archivo por día durante 3 días.                                                                                                                                  |

### 3.3 Columnas del CSV generado por Java

| Columna      | Descripción                         | Origen                        |
|--------------|-------------------------------------|-------------------------------|
| IP Origen    | Dirección IP del emisor del paquete | Cabecera IPv4                 |
| IP Destino   | Dirección IP del receptor           | Cabecera IPv4                 |
| MAC Origen   | Dirección MAC del emisor (o N/A)    | Cabecera Ethernet             |
| MAC Destino  | Dirección MAC del receptor (o N/A)  | Cabecera Ethernet             |
| Puerto Ori   | Puerto de origen (TCP/UDP)          | Cabecera TCP o UDP            |
| Puerto Des   | Puerto de destino (TCP/UDP)         | Cabecera TCP o UDP            |
| Protocolo    | TCP o UDP                           | Tipo de paquete               |
| Longitud     | Tamaño del paquete en bytes         | <code>pkt.length()</code>     |
| Hora Captura | Hora en formato HH:mm:ss            | Reloj del sistema al capturar |

| Columna      | Descripción                               | Origen                                                                 |
|--------------|-------------------------------------------|------------------------------------------------------------------------|
| Equipo Local | Nombre del equipo donde corre el programa | Campo de texto / <code>InetAddress.getLocalHost().getHostName()</code> |
| SSID         | Nombre de la red Wi-Fi                    | Campo de texto (manual)                                                |

## 4. Parte 2: Análisis en Python

Todo el análisis está en la carpeta `analisis_python/`. Los scripts asumen que el CSV está en la raíz del proyecto (`../captura_red.csv`) o que se pasa la ruta al llamar a `cargar_datos(ruta_csv=...)`. También se puede usar `captura_red_YYYY-MM-DD.csv` si se indica la ruta correcta.

### 4.1 Archivo: `analisis_python/limpieza_datos.py`

**Propósito:** Cargar el CSV, limpiarlo y dejarlo listo para modelos y gráficas.

Conceptos que usa:

- **DataFrame (pandas):** Estructura tabular en memoria (filas y columnas). Todas las operaciones de limpieza y derivación se hacen sobre un `pd.DataFrame`.
- **Limpieza:** Eliminación de duplicados, filas con valores nulos en columnas clave (IPs, puertos, protocolo) y filtrado para conservar solo TCP y UDP.
- **Tipos de datos:** Conversión de puertos y longitud a numérico; construcción de un campo datetime a partir de la hora de captura (con una fecha base) para poder extraer la "hora del día" (0–23).
- **Ingeniería de características:** Creación de columnas nuevas a partir de las existentes:
  - **servicio:** Categoría según puerto destino (HTTPS, DNS, HTTP, etc.) usando un diccionario de puertos conocidos.
  - **rango\_puerto\_dst / rango\_puerto\_src:** Clasificación del puerto en "Conocido" (<1024), "Registrado" (1024–49151) o "Dinámico" (>49151).
  - **app\_inferida:** Heurística por **prefijo de IP destino** para asociar aproximadamente a un servicio (Google/YouTube, Meta, Microsoft, Cloudflare, etc.). Es orientativo, no exacto.
- **Codificación:** `LabelEncoder` transforma categorías (por ejemplo Protocolo, Servicio) en números para que los algoritmos puedan usarlas.
- **Normalización:** `StandardScaler` deja media 0 y desviación típica 1 en columnas numéricas (longitud, puertos), necesario para KNN y K-Means.

**Función principal:** `cargar_datos(ruta_csv, verbose)`. Devuelve el DataFrame con todas las columnas originales y las derivadas (incluidas `longitud_scaled`, `po_scaled`, `pd_scaled`, `protocolo_enc`, `servicio_enc`, `hora`, `app_inferida`).

### 4.2 Archivo: `analisis_python/modelos.py`

**Propósito:** Implementar los tres modelos pedidos en la práctica.

#### Modelo 1 — Clasificación KNN (K-Nearest Neighbors)

- **Función:** `clasificar_tipo_trafico(df, k=5, test_size=0.2)`.

- **Qué hace:** Usa como características `longitud_scaled`, `po_scaled`, `pd_scaled`, `protocolo_enc` y como etiqueta la categoría de **servicio** (HTTPS, DNS, etc.). Divide los datos en entrenamiento (80 %) y prueba (20 %), entrena un `KNeighborsClassifier` con `k=5` vecinos y calcula accuracy e informe de clasificación. Devuelve un diccionario con el modelo, predicciones, matriz de confusión y nombres de clases.
- **Uso en la práctica:** Clasificar el "tipo de tráfico" (servicio de red) de cada paquete a partir de sus atributos numéricos y del protocolo.

## Modelo 2 — Clustering K-Means

- **Función:** `clustering_equipos(df, k=3)`.
- **Qué hace:** Primero agrega los datos por **IP de origen** (total de paquetes, bytes totales, bytes promedio, puertos únicos, porcentaje TCP). Sobre esas métricas aplica `StandardScaler` y luego **K-Means** con `k=3`. Asigna a cada equipo un cluster y lo etiqueta como "Intensivo", "Moderado" o "Ligero" según el volumen de tráfico. Usa **PCA** con 2 componentes para obtener coordenadas 2D y poder dibujar el scatter de clusters. Calcula el **silhouette score** para evaluar la calidad del agrupamiento.
- **Uso en la práctica:** Agrupar equipos (IPs) por "nivel de uso" de la red sin etiquetas previas.

## Modelo 3 — Análisis predictivo (regresión logística)

- **Función:** `predecir_protocolo_por_hora(df)`.
- **Qué hace:** Usa como características la **hora del día** (0–23) y la longitud del paquete normalizada, y como etiqueta el **protocolo** (TCP/UDP codificado). Entrena una `LogisticRegression`, calcula accuracy e informe de clasificación y genera una tabla "predicción por hora" con la probabilidad de TCP/UDP para cada hora. Devuelve el modelo, accuracy, predicciones y esa tabla.
- **Uso en la práctica:** Estimar qué protocolo es más probable en cada hora del día.

### 4.3 Archivo: `analisis_python/visualizacion.py`

**Propósito:** Generar todas las gráficas en formato PNG y guardarlas en `analisis_python/resultados/`.

Usa **matplotlib** y **seaborn**. Cada función corresponde a una gráfica; al final se llama `_guardar(nombre)` para guardar y cerrar la figura. La función `generar_todas(df, res_knn, res_cluster, res_pred)` invoca en secuencia todas las gráficas (TCP/UDP, dispersión, puertos, clusters, tráfico por hora, matriz de confusión KNN, predicción por hora, servicios, apps inferidas).

### 4.4 Archivo: `analisis_python/main.py`

**Propósito:** Ejecutar el flujo completo **sin Jupyter**.

1. Llama a `cargar_datos(verbose=True)`.
2. Llama a `clasificar_tipo_trafico`, `clustering_equipos` y `predecir_protocolo_por_hora`.
3. Llama a `generar_todas` con el DataFrame y los resultados de los tres modelos.
4. Imprime un resumen (registros, accuracy KNN, silhouette, accuracy predictivo) y recuerda que las gráficas están en `analisis_python/resultados/`.

### 4.5 Archivo: `analisis_python/analisis_trafico.ipynb`

Es el **notebook Jupyter** que reproduce el mismo flujo por secciones (carga, exploración, modelos, visualizaciones, conclusiones). Las gráficas pueden mostrarse dentro del notebook si se ejecutan las celdas. Es opcional; el análisis completo ya se hace con `main.py`.

---

## 5. Técnicas de Data Science utilizadas

### 5.1 KNN (K-Nearest Neighbors)

- **Concepto:** Algoritmo de **clasificación supervisada**. Para clasificar un registro nuevo, busca los **k** registros más cercanos en el espacio de características y asigna la clase mayoritaria entre esos vecinos. La "distancia" se calcula normalmente en las variables ya escaladas.
- **En este proyecto:** Se usa para predecir la **categoría de servicio** (HTTPS, DNS, etc.) de cada paquete a partir de longitud, puertos y protocolo. El número de vecinos es **k=5**.

### 5.2 K-Means (clustering)

- **Concepto:** Algoritmo de **agrupación no supervisada**. Divide los datos en **k** grupos (clusters) de forma que los puntos dentro de un grupo sean parecidos entre sí y distintos de los de otros grupos. No usa etiquetas; solo las características.
- **En este proyecto:** Se agrupan **equipos** (agregados por IP origen) según volumen de tráfico, bytes y puertos. Se usa **k=3** para obtener tres niveles: Intensivo, Moderado, Ligero.

### 5.3 Regresión logística

- **Concepto:** Modelo de **clasificación** que estima la probabilidad de pertenecer a una clase (por ejemplo TCP vs UDP) usando una función logística. Aunque se llama "regresión", en la práctica se usa para clasificación binaria o multiclase.
- **En este proyecto:** Predice el **protocolo predominante (TCP o UDP)** en función de la hora del día y del tamaño del paquete.

### 5.4 Otras nociones

- **Train/test split:** División de los datos en conjunto de entrenamiento (para ajustar el modelo) y de prueba (para evaluar). En los modelos se usa típicamente 80 % train, 20 % test.
  - **Accuracy:** Proporción de predicciones correctas sobre el total.
  - **Matriz de confusión:** Tabla que cruza clase real vs clase predicha; permite ver aciertos y tipos de error.
  - **Silhouette score:** Métrica entre -1 y 1 que mide qué tan bien separados están los clusters; valores más altos indican mejor agrupación.
  - **PCA (Principal Component Analysis):** Técnica de reducción de dimensión que obtiene nuevas variables (componentes principales) para poder visualizar en 2D datos que tienen más dimensiones (por ejemplo los 5 indicadores por equipo en K-Means).
  - **LabelEncoder / StandardScaler:** Herramientas de scikit-learn para codificar categorías y escalar variables numéricas antes de entrenar los modelos.
- 

## 6. Gráficas generadas

Todas se guardan en [analisis\\_python/resultados/](#) con los nombres indicados. Se generan a partir del CSV exportado por NetworkMonitor (Java).

| Archivo                                          | Descripción breve                                                                                     |
|--------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| <a href="#">01_tcp_vs_udp.png</a>                | Barras y gráfico de pastel: cantidad y proporción de paquetes TCP vs UDP.                             |
| <a href="#">02_dispersion_tamano_hora.png</a>    | Dispersión: tamaño del paquete (eje Y) vs hora del día (eje X), diferenciando TCP y UDP.              |
| <a href="#">03_puertos_frecuentes.png</a>        | Barras horizontales: los 15 puertos destino más usados.                                               |
| <a href="#">04_clusters_equipos.png</a>          | Scatter 2D (PCA): cada punto es un equipo; color = cluster (Intensivo/Moderado/Ligero).               |
| <a href="#">05_trafico_por_hora.png</a>          | Líneas: número de paquetes por hora del día, por protocolo.                                           |
| <a href="#">06_confusion_knn.png</a>             | Heatmap: matriz de confusión del clasificador KNN (servicios).                                        |
| <a href="#">07_prediccion_protocolo_hora.png</a> | Áreas/líneas: probabilidad de TCP y UDP por hora (modelo predictivo).                                 |
| <a href="#">08_servicios_frecuentes.png</a>      | Barras: categorías de servicio (por puerto) más frecuentes.                                           |
| <a href="#">09_apps_inferidas.png</a>            | Barras y pastel: aplicaciones/servicios inferidos por prefijo de IP (Google, Meta, Cloudflare, etc.). |

## 7. Flujo completo de datos

1. Usuario ejecuta NetworkMonitor (Java).
2. Selecciona interfaz Wi-Fi e inicia la captura.
3. Pcap4J recibe paquetes; processPacket() extrae IP, MAC, puertos, protocolo, longitud, hora.
4. Cada paquete válido (IPv4 + TCP o UDP) se añade como fila en la tabla (DefaultTableModel).
5. Usuario puede filtrar la vista (protocolo, IP, puerto) con TableRowSorter.
6. Al pulsar "Exportar CSV", se escribe captura\_red\_YYYY-MM-DD.csv con encabezados y todas las filas.
7. Usuario ejecuta py main.py (o el notebook) en analisis\_python/.
8. limpieza\_datos.cargar\_datos() lee el CSV, limpia, deriva columnas (servicio, hora, app\_inferida), codifica y escala.
9. modelos.py entrena KNN, K-Means y regresión logística sobre ese DataFrame.
10. visualizacion.generar\_todas() produce las 9 gráficas en resultados/.
11. CONCLUSIONES.md y EVIDENCIA.md documentan resultados y entrega.

## 8. Cómo ejecutar todo

Java (captura)

1. Abrir el proyecto en **Apache NetBeans**.
2. Tener **Npcap** instalado (Windows) o libpcap (Linux/Mac).
3. Tener la librería **Pcap4J** en el proyecto (según [pom.xml](#) o librerías de NetBeans).
4. Ejecutar la clase **NetworkMonitor**. Seleccionar interfaz, capturar y exportar CSV cuando se desee.

## Python (análisis, sin Jupyter)

1. En terminal: `cd analisis_python`
2. `py -m pip install -r requirements.txt`
3. Asegurarse de que exista un CSV en la ruta por defecto (por ejemplo `../captura_red.csv`) o pasar la ruta al llamar a `cargar_datos(ruta_csv="ruta/al/archivo.csv")`.
4. `py main.py`
5. Revisar las gráficas en `analisis_python/resultados/` y el texto en consola.

## Python (con Jupyter)

1. Instalar: `py -m pip install jupyter`
2. `cd analisis_python` y `jupyter notebook analisis_trafico.ipynb`
3. Ejecutar las celdas en orden; las gráficas pueden mostrarse dentro del notebook.

---

## 9. Glosario de términos

| Término                    | Significado                                                                                                                      |
|----------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| <b>CSV</b>                 | Archivo de texto con valores separados por comas; fácil de leer en Excel, pandas o WEKA.                                         |
| <b>DataFrame</b>           | Estructura tabular de pandas (filas y columnas) para análisis en Python.                                                         |
| <b>DPI</b>                 | Deep Packet Inspection; inspección profunda del contenido del paquete (no implementada aquí).                                    |
| <b>Encabezado (header)</b> | Parte inicial de un paquete o trama donde van direcciones, puertos, protocolo, etc.                                              |
| <b>Ethernet</b>            | Tecnología de capa 2; la cabecera Ethernet contiene las direcciones MAC.                                                         |
| <b>HTTPS</b>               | HTTP sobre TLS; tráfico web cifrado, normalmente en puerto 443.                                                                  |
| <b>IPv4</b>                | Versión 4 del protocolo IP; direcciones de 32 bits (ej. 10.20.53.78).                                                            |
| <b>Modo promiscuo</b>      | Modo en el que la tarjeta de red captura todo el tráfico que ve, no solo el dirigido a ella.                                     |
| <b>SNI</b>                 | Server Name Indication; campo en TLS que indica el dominio; permitiría identificar sitio/app pero no se extrae en este proyecto. |
| <b>SSID</b>                | Nombre de la red Wi-Fi.                                                                                                          |
| <b>WEKA</b>                | Herramienta de minería de datos que puede importar CSV; el CSV generado es compatible.                                           |

---

*Documentación generada para el proyecto NetworkMonitor — Práctica de monitoreo y análisis de tráfico de red con Data Science.*